Deliverable 2: a working supervised agent

Individual · 10% of course · Due November 6, 2026, 23:59 Toronto time

Extend the D1 repository. Correct defects identified in D1 feedback and identify the corrections in `CHANGELOG.md`. Do not start an unrelated product. The overview's policy version 1 remains in force.

Required work

D2.1 — Model and tool loop. Implement the four-tool protocol, validated dispatch and observation history. The runner must perform successive model calls informed by earlier tool results. Enforce the 1–6 call limit, count invalid outputs, and return controlled outcomes for malformed requests, unknown tools, timeouts and exhausted attempts. Keep a replay mode and a distinct model-connected mode. The replay provider is a test double, not the AI model.

D2.2 — Approval and execution. A proposal stops at pending approval. A separate operator action approves or rejects it. Before execution, recheck booking version, policy version, current target eligibility and proposal identity. Preserve unchanged domain state after rejection. A repeated committed proposal returns a duplicate outcome without a second commit or notification record. Enforce these controls in Java; a prompt alone is not a control.

D2.3 — Periodic task detection. Implement explicit 60-second ticks with controllable time. Detect bookings on closed spaces, suppress repeated work for the same booking/policy versions, and avoid issuing another active proposal for the same state. Do not use sleep to make the assessor wait. Demonstrate boundary ticks and a backwards-clock rejection. The overview specifies the conservative retry policy.

D2.4 — Design. Separate model protocol details, application coordination, domain rules and data storage through appropriate boundaries. Apply patterns only where they solve a stated problem; explain at least two significant design choices and a considered alternative. A named pattern is not mandatory for each choice. Update class and sequence diagrams; add a proposal-state diagram. Do not count interfaces or patterns as a quality score.

D2.5 — Testing and model demonstration. Preserve D1 regression tests. Add the cases below with explicit state assertions. Include a real model transcript when supported access is available. Apply the overview's access-contingency rule if the course endpoint has not been supplied; do not buy access as a workaround. The connection code and deterministic contract tests remain assessable regardless of service availability.

Required acceptance cases

Case	Stimulus	Expected result
D2-A	Replay: read, list, propose B12	One pending proposal; no automatic booking change
D2-B	Execute pending proposal	NOT_APPROVED; all protected state unchanged
D2-C	Approve then execute valid proposal	One COMMITTED result, B1 at B12/version 1, one simulated notification
D2-D	Execute same committed proposal again	DUPLICATE; commit/notification counts unchanged
D2-E	Reject pending proposal then execute	No commit; booking unchanged
D2-F	Change booking version after proposal	STALE; no move
D2-G	Close/occupy target after proposal	UNAVAILABLE_OR_INELIGIBLE; no move
D2-H	Malformed JSON or extra `approved` field	Request rejected; attempt counted; no authority granted

Case	Stimulus	Expected result
D2-I	Unknown tool or wrong goal booking	Controlled tool rejection; no arbitrary operation
D2-J	Six invalid responses	STEP_LIMIT after exactly six calls
D2-K	Provider throws/times out	PROVIDER_ERROR; no commit
D2-L	Ticks 0, 59, 60 with unchanged versions	At most one automatically emitted work item
D2-M	Tick below previous time	Clear clock rejection
D2-N	Real model run through allowed tools	Genuine transcript and permitted result; or documented course-access contingency

A provider may choose C03 instead of B12 under policy 1; that is also eligible. Do not require a model to reproduce the scripted target or exact natural-language text. Stale and rejected outcomes must check state, not only a printed label.

Report and submission additions

`DESIGN.pdf` is at most five pages plus readable diagrams. Include a component/dependency explanation, the state transition table, two design decisions, one unsuccessful or rejected model/tool interaction and a limitations section. In `results/`, retain deterministic check output, JUnit output, the periodic trace and a redacted live transcript or the explicit pending-access status. Provide `run.sh run scripted` and a documented live command; neither should commit automatically in live mode without the separate operator action.

Rubric: 100 release points

Criterion	Points	Full-credit expectation
Model/tool loop and provider boundary	20	Successive observations, strict dispatch, bounded execution, genuine model integration route
Approval, current-state and duplicate controls	25	All required rejected/committed paths; unchanged-state checks and one-commit semantics
Periodic work detection	10	Controllable time, boundary logic and duplicate suppression
Architecture and justified design choices	15	Useful separation, coherent dependencies and consistent diagrams
Automated failure/regression testing	20	Required cases and meaningful assertions; real command results
Reproducibility, model record and explanation	10	Repeatable modes, truthful provenance and concise design rationale

The course-access contingency does not remove the model/provider boundary requirement. The missing live-run record is replaced temporarily by the supported protocol/fixture demonstration and completed in the announced course-supported session. The model's choice of any eligible alternative does not reduce marks. The quality of an expensive model does not earn extra marks.

Submission and individual authorship

This is an individual project. Submit your own repository snapshot and explain every included design decision. Do not submit a shared implementation, team report or another student's source. Discussion of general concepts is permitted; exchange of assessed code, diagrams or solutions is not. Designated AI

assistance is permitted with the disclosure below. Supplied infrastructure is identified explicitly and may be retained with attribution.

Submit one ZIP through the named eClass assignment before **23:59 Toronto time** on the stated date. Name it `EECS3311_Dn_STUDENTNUMBER.zip`, replacing `Dn` and `STUDENTNUMBER`. Its single top-level folder must contain `src/`, `test/`, `run.sh`, `README.md`, `DESIGN.pdf`, `AI_USE.md`, `TEST_PLAN.md`, `CHANGELOG.md` and `results/`. Keep generated `.class` files, IDE caches, model weights, secrets and downloaded libraries out. A private Git repository is recommended for development; the submitted ZIP is the marking snapshot. Do not rely on a remote branch changing after the deadline. Keep a copy of the uploaded receipt and ZIP.

`README.md` must state the JDK, exact execution commands, supported scenario(s), known limitations and any permitted starter files reused. Both `bash run.sh check` and the appropriate prepared demonstration must run from the extracted project root. Jupiter tests use `bash setup-junit.sh` once on the supported network, then `bash run.sh test`; the TA will provide/cache the pinned dependency. Include genuine command output with date, command and environment in `results/`. Do not label a plain-Java check run as a JUnit run or label replay as a live-model run.

In `AI_USE.md`, identify each tool/model used, date or version when known, the task it helped with, the source files or diagrams affected, what you changed or rejected, and how you checked the result. State "No generative AI assistance used" when applicable. Whole personal chat histories are not required. Remove credentials and personal information from any excerpt. Code produced with assistance remains your responsibility. The instructor may ask you to explain or make a small change to your submitted work under the course's academic-integrity procedures; there is no separate undisclosed viva mark.

Submit even if one part is incomplete: identify the affected requirement precisely. Do not hide a failure by deleting the test or editing a captured result. Required extensions cannot be replaced by a polished user interface. Optional features earn no substitute marks for missing core requirements. Approved accommodations and extensions take precedence over ordinary deadlines; request assistance through eClass promptly. Late/absence arrangements are governed by the published syllabus and approved course procedures, not by this brief inventing a separate penalty.